

LinksPlatform's Platform.Interfaces Class Library

0.1 csharp/Platform.Interfaces.Tests/InterfacesTests.cs

```
1  using System.IO;
2  using System.Reflection.PortableExecutable;
3  using Xunit;
4
5  namespace Platform.Interfaces.Tests
6  {
7      public static class InterfacesTests
8      {
9          [Fact]
10         public static void BuildTest()
11         {
12             ICounter<int, int>? c1 = null;
13             ICounter<int>? c2 = null;
14             IMatcher<int>? cm1 = null;
15             IFactory<int>? f1 = null;
16             IProperties<int, int, int>? p1 = null;
17             IProperty<int, int>? p2 = null;
18             IProvider<int, int>? p3 = null;
19             IProvider<int>? p4 = null;
20             ISetter<int, int>? s1 = null;
21             ISetter<int>? s2 = null;
22
23             Assert.Null(c1);
24             Assert.Null(c2);
25             Assert.Null(cm1);
26             Assert.Null(f1);
27             Assert.Null(p1);
28             Assert.Null(p2);
29             Assert.Null(p3);
30             Assert.Null(p4);
31             Assert.Null(s1);
32             Assert.Null(s2);
33         }
34
35         [Fact]
36         public static void MatcherTestsCandidate()
37         {
38             IMatcher<int> matcher = new MinimumMatcher(10);
39
40             Assert.False(matcher.IsMatched(9));
41             Assert.True(matcher.IsMatched(10));
42         }
43
44         [Fact]
45         public static void AssemblyContainsEmbeddedPortablePdb()
46         {
47             using var assembly = File.OpenRead(typeof(ICounter<>).Assembly.Location);
48             using var peReader = new PEReader(assembly);
49
50             Assert.Contains(
51                 peReader.ReadDebugDirectory(),
52                 entry => entry.Type == DebugDirectoryEntryType.EmbeddedPortablePdb);
53         }
54
55         private sealed class MinimumMatcher(int minimum) : IMatcher<int>
56         {
57             public bool IsMatched(int candidate) => candidate >= minimum;
```

```

58     }
59 }
60 }

```

0.2 csharp/Platform.Interfaces/ICli.cs

```

1  namespace Platform.Interfaces;
2
3  /// <summary>
4  /// <para>Defines command line interfaces for command that interacts with an operating
   ↪ system.</para>
5  /// <para>Определяет интерфейс командной строки, для команды взаимодействующей с операционной
   ↪ системой.</para>
6  /// </summary>
7  public interface ICli
8  {
9      /// <summary>
10     /// <para>Runs a command.</para>
11     /// <para>Запускает команду.</para>
12     /// </summary>
13     /// <param name="args">
14     /// <para>Arguments for a command.</para>
15     /// <para>Аргументы для команды.</para>
16     /// </param>
17     /// <returns>
18     /// <para>Returns command's exit code.</para>
19     /// <para>Возвращает код выхода команды.</para>
20     /// </returns>
21     int Run(params string[] args);
22 }

```

0.3 csharp/Platform.Interfaces/ICounter[TResult, TArgument].cs

```

1  namespace Platform.Interfaces
2  {
3      /// <summary>
4      /// <para>Defines a counter that requires an argument to perform a count.</para>
5      /// <para>Определяет счётчик, которому требуется аргумент для выполнения подсчёта.</para>
6      /// </summary>
7      /// <typeparam name="TArgument">
8      /// <para>The argument type.</para>
9      /// <para>Тип аргумента.</para>
10     /// </typeparam>
11     /// <typeparam name="TResult">
12     /// <para>The count result type.</para>
13     /// <para>Тип результата подсчёта.</para>
14     /// </typeparam>
15     public interface ICounter<out TResult, in TArgument>
16     {
17         /// <summary>
18         /// <para>Performs a count.</para>
19         /// <para>Выполняет подсчёт.</para>
20         /// </summary>
21         /// <param name="argument">
22         /// <para>The argument.</para>
23         /// <para>Аргумент.</para>
24         /// </param>
25         /// <returns>
26         /// <para>The count result.</para>
27         /// <para>Результат подсчёта.</para>
28         /// </returns>
29         TResult Count(TArgument argument);

```

```

30     }
31 }

```

0.4 csharp/Platform.Interfaces/ICounter[TResult].cs

```

1 namespace Platform.Interfaces
2 {
3     /// <summary>
4     /// <para>Defines a counter.</para>
5     /// <para>Определяет счётчик.</para>
6     /// </summary>
7     /// <typeparam name="TResult">
8     /// <para>The count result type.</para>
9     /// <para>Тип результата подсчёта.</para>
10    /// </typeparam>
11    public interface ICounter<out TResult>
12    {
13        /// <summary>
14        /// <para>Performs a count.</para>
15        /// <para>Выполняет подсчёт.</para>
16        /// </summary>
17        /// <returns>
18        /// <para>The count result.</para>
19        /// <para>Результат подсчёта.</para>
20        /// </returns>
21        TResult Count();
22    }
23 }

```

0.5 csharp/Platform.Interfaces/IFactory.cs

```

1 namespace Platform.Interfaces
2 {
3     /// <summary>
4     /// <para>Defines a factory that produces instances of a specific type.</para>
5     /// <para>Определяет фабрику, которая производит экземпляры определенного типа.</para>
6     /// </summary>
7     /// <typeparam name="TProduct">
8     /// <para>Type of produced instances.</para>
9     /// <para>Тип производимых экземпляров.</para>
10    /// </typeparam>
11    public interface IFactory<out TProduct>
12    {
13        /// <summary>
14        /// <para>Creates an instance of <typeparamref name="TProduct"/> type.</para>
15        /// <para>Создает экземпляр типа <typeparamref name="TProduct"/>.</para>
16        /// </summary>
17        /// <returns>
18        /// <para>The instance of <typeparamref name="TProduct"/> type.</para>
19        /// <para>Экземпляр типа <typeparamref name="TProduct"/>.</para>
20        /// </returns>
21        TProduct Create();
22    }
23 }

```

0.6 csharp/Platform.Interfaces/IMatcher.cs

```

1 namespace Platform.Interfaces
2 {
3     /// <summary>
4     /// <para>Defines a matcher that determines whether a candidate satisfies its matching
5     ↪ rule.</para>
6     /// <para>Определяет объект, который проверяет, соответствует ли кандидат правилу
7     ↪ сопоставления.</para>

```

```

6     /// </summary>
7     /// <typeparam name="TCandidate">
8     /// <para>Type of the value being tested for a match.</para>
9     /// <para>Тип значения, проверяемого на соответствие.</para>
10    /// </typeparam>
11    public interface IMatcher<in TCandidate>
12    {
13        /// <summary>
14        /// <para>Determines whether the candidate satisfies the matching rule.</para>
15        /// <para>Определяет, соответствует ли кандидат правилу сопоставления.</para>
16        /// </summary>
17        /// <param name="candidate">
18        /// <para>The value to test.</para>
19        /// <para>Проверяемое значение.</para>
20        /// </param>
21        /// <returns>
22        /// <para>Whether the candidate satisfies the matching rule.</para>
23        /// <para>Соответствует ли кандидат правилу сопоставления.</para>
24        /// </returns>
25        bool IsMatched(TCandidate candidate);
26    }
27 }

```

0.7 csharp/Platform.Interfaces/IProperties.cs

```

1 namespace Platform.Interfaces
2 {
3     /// <summary>
4     /// <para>Defines a properties operator that is able to get or set values of properties of a
5     ↪ object of a specific type.</para>
6     /// <para>Определяет оператор свойств, который может получать или устанавливать значения
7     ↪ свойств объекта определенного типа.</para>
8     /// </summary>
9     /// <typeparam name="TObject">
10    /// <para>Object type.</para>
11    /// <para>Тип объекта.</para>
12    /// </typeparam>
13    /// <typeparam name="TProperty">
14    /// <para>Property reference type.</para>
15    /// <para>Тип ссылки на свойство.</para>
16    /// </typeparam>
17    /// <typeparam name="TValue">
18    /// <para>Property value type.</para>
19    /// <para>Тип значения свойства.</para>
20    /// </typeparam>
21    public interface IProperties<in TObject, in TProperty, TValue>
22    {
23        /// <summary>
24        /// <para>Gets the value of the property in the specified object.</para>
25        /// <para>Получает значение свойства в указанном объекте.</para>
26        /// </summary>
27        /// <param name="object">
28        /// <para>The object reference.</para>
29        /// <para>Ссылка на объект.</para>
30        /// </param>
31        /// <param name="property">
32        /// <para>The property reference.</para>
33        /// <para>Ссылка на свойство.</para>
34        /// </param>
35        /// <returns>
36        /// <para>The value of the property.</para>

```

```

35     /// <para>Значение свойства.</para>
36     /// </returns>
37     TValue GetValue(TObject @object, TProperty property);
38
39     /// <summary>
40     /// <para>Sets the value of a property in the specified object.</para>
41     /// <para>Устанавливает значение свойства в указанном объекте.</para>
42     /// </summary>
43     /// <param name="object">
44     /// <para>The object reference.</para>
45     /// <para>Ссылка на объект.</para>
46     /// </param>
47     /// <param name="property">
48     /// <para>The property reference.</para>
49     /// <para>Ссылка на свойство.</para>
50     /// </param>
51     /// <param name="value">
52     /// <para>The value.</para>
53     /// <para>Значение.</para>
54     /// </param>
55     void SetValue(TObject @object, TProperty property, TValue value);
56 }
57 }

```

0.8 csharp/Platform.Interfaces/IProperty.cs

```

1  namespace Platform.Interfaces
2  {
3      /// <summary>
4      /// <para>Defines a specific property operator that is able to get or set values of that
5      ↪ property.</para>
6      /// <para>Определяет оператор определённого свойства, который может получать или
7      ↪ устанавливать его значения.</para>
8      /// </summary>
9      /// <typeparam name="TObject">
10     /// <para>Object type.</para>
11     /// <para>Тип объекта.</para>
12     /// </typeparam>
13     /// <typeparam name="TValue">
14     /// <para>Property value type.</para>
15     /// <para>Тип значения свойства.</para>
16     /// </typeparam>
17     public interface IProperty<in TObject, TValue> : ISetter<TValue, TObject>, IProvider<TValue,
18     ↪ TObject>
19     {
20     }
21 }

```

0.9 csharp/Platform.Interfaces/IProvider[TProvided, TArgument].cs

```

1  namespace Platform.Interfaces
2  {
3      /// <summary>
4      /// <para>Defines the provider of objects/values for which an argument must be
5      ↪ specified.</para>
6      /// <para>Определяет поставщика объектов/значений, для получения которых необходимо указать
7      ↪ аргумент.</para>
8      /// </summary>
9      /// <typeparam name="TProvided">
10     /// <para>Type of provided objects/values.</para>
11     /// <para>Тип предоставляемых объектов/значений.</para>
12     /// </typeparam>

```

```

11     /// <typeparam name="TArgument">
12     /// <para>Argument type.</para>
13     /// <para>Тип аргумента.</para>
14     /// </typeparam>
15     public interface IProvider<out TProvided, in TArgument>
16     {
17         /// <summary>
18         /// <para>Provides an object(s)/value(s).</para>
19         /// <para>Предоставляет объект(ы)/значение(я).</para>
20         /// </summary>
21         /// <param name="argument">
22         /// <para>The argument required to acquire the object(s)/value(s).</para>
23         /// <para>Аргумент, необходимый для получения объекта(ов)/значения(ий).</para>
24         /// </param>
25         /// <returns>
26         /// <para>The object(s)/value(s).</para>
27         /// <para>Объект(ы)/значение(я).</para>
28         /// </returns>
29         TProvided Get(TArgument argument);
30     }
31 }

```

0.10 csharp/Platform.Interfaces/IProvider[TProvided].cs

```

1 namespace Platform.Interfaces
2 {
3     /// <summary>
4     /// <para>Defines the provider of objects/values.</para>
5     /// <para>Определяет поставщика объектов/значений.</para>
6     /// </summary>
7     /// <typeparam name="TProvided">
8     /// <para>Type of provided object/value.</para>
9     /// <para>Тип предоставляемого объекта/значения.</para>
10    /// </typeparam>
11    public interface IProvider<out TProvided>
12    {
13        /// <summary>
14        /// <para>Provides an object(s)/value(s).</para>
15        /// <para>Предоставляет объект(ы)/значение(я).</para>
16        /// </summary>
17        /// <returns>
18        /// <para>The object(s)/value(s).</para>
19        /// <para>Объект(ы)/значение(я).</para>
20        /// </returns>
21        TProvided Get();
22    }
23 }

```

0.11 csharp/Platform.Interfaces/ISetter[TValue, TArgument].cs

```

1 namespace Platform.Interfaces
2 {
3     /// <summary>
4     /// <para>Defines an setter that requires an argument to set the passed value as a new
5     ↪ state.</para>
6     /// <para>Определяет установщик, которому для установки переданного значения в качестве
7     ↪ нового состояния требуется аргумент.</para>
8     /// </summary>
9     /// <typeparam name="TValue">
10    /// <para>Type of set value.</para>
11    /// <para>Тип устанавливаемого значения.</para>
12    /// </typeparam>

```

```

11  /// <typeparam name="TArgument">
12  /// <para>The argument type.</para>
13  /// <para>Тип аргумента.</para>
14  /// </typeparam>
15  public interface ISetter<in TValue, in TArgument>
16  {
17      /// <summary>
18      /// <para>Sets the value of a specific property in the specified object.</para>
19      /// <para>Устанавливает значение определённого свойства в указанном объекте.</para>
20      /// </summary>
21      /// <param name="argument">
22      /// <para>The argument.</para>
23      /// <para>Аргумент.</para>
24      /// </param>
25      /// <param name="value">
26      /// <para>The value.</para>
27      /// <para>Значение.</para>
28      /// </param>
29      void Set(TArgument argument, TValue value);
30  }
31  }

```

0.12 csharp/Platform.Interfaces/ISetter[TValue].cs

```

1  namespace Platform.Interfaces
2  {
3      /// <summary>
4      /// <para>Defines an setter that sets the passed value as a new state.</para>
5      /// <para>Определяет установщик, который устанавливает переданное значение в качестве нового
6      /// ↪ состояния.</para>
7      /// </summary>
8      /// <typeparam name="TValue">
9      /// <para>Type of set value.</para>
10     /// <para>Тип устанавливаемого значения.</para>
11     /// </typeparam>
12     public interface ISetter<in TValue>
13     {
14         /// <summary>
15         /// <para>Sets the value of a specific property in the specified object.</para>
16         /// <para>Устанавливает значение определённого свойства в указанном объекте.</para>
17         /// </summary>
18         /// <param name="value">
19         /// <para>The value.</para>
20         /// <para>Значение.</para>
21         /// </param>
22         void Set(TValue value);
23     }
24 }

```

Index

csharp/Platform.Interfaces.Tests/InterfacesTests.cs, 1
csharp/Platform.Interfaces/ICli.cs, 2
csharp/Platform.Interfaces/ICounter[TResult, TArgument].cs, 2
csharp/Platform.Interfaces/ICounter[TResult].cs, 3
csharp/Platform.Interfaces/IFactory.cs, 3
csharp/Platform.Interfaces/IMatcher.cs, 3
csharp/Platform.Interfaces/IProperties.cs, 4
csharp/Platform.Interfaces/IProperty.cs, 5
csharp/Platform.Interfaces/IProvider[TProvided, TArgument].cs, 5
csharp/Platform.Interfaces/IProvider[TProvided].cs, 6
csharp/Platform.Interfaces/ISetter[TValue, TArgument].cs, 6
csharp/Platform.Interfaces/ISetter[TValue].cs, 7